PHASE

8

Processes, Context Switching and Scheduling

Run many things at once on one CPU: kernel threads, the context switch, and a preemptive scheduler driven by the timer

Depends on Phase 7: `kmalloc`, `list.h`, `make test`; Phase 4: the timer interrupt

Delivers Kernel threads, each with its own stack and guard page, a context switch in about ten lines of assembly, cooperative `yield`, then preemptive round-robin scheduling on the timer, `sleep_ms` with proper blocking, an idle task that halts the CPU, task exit with reaping, and a `ps` command. Optionally, a fair scheduler in the style of Linux's CFS.

1 Scope and motivation

This is where Kagan stops being a program and becomes an operating system. A single CPU can only run one instruction stream at a time, but by switching between streams quickly enough, the kernel creates the illusion that many run at once. OSTEP calls this *virtualizing the CPU*, and it is the first of its three big ideas.

Kagan builds it in two layers, as the textbooks do. The **mechanism** is the context switch: saving everything that describes one running thread and restoring another. The **policy** is the scheduler: deciding which thread runs next and for how long. The first tasks are kernel threads, which all share the kernel's address space; in Phase 10 the same machinery switches between user processes with separate address spaces. The context switch is short and subtle, and every later phase depends on it.

2 Background

2.1 The process abstraction

OSTEP chapter 4 defines a process by its **machine state**: the registers (including the instruction and stack pointers), the memory it can address, and the I/O resources it holds. The kernel keeps this state in a **process control block** (PCB), which Kagan calls `struct task`, and tracks every task's **state** (Figure 1). A task is *running* on the CPU, *ready* to run and waiting its turn, *blocked* waiting for an event (a timer, a key, a disk block), or a *zombie* that has finished but whose resources have not been reclaimed yet.

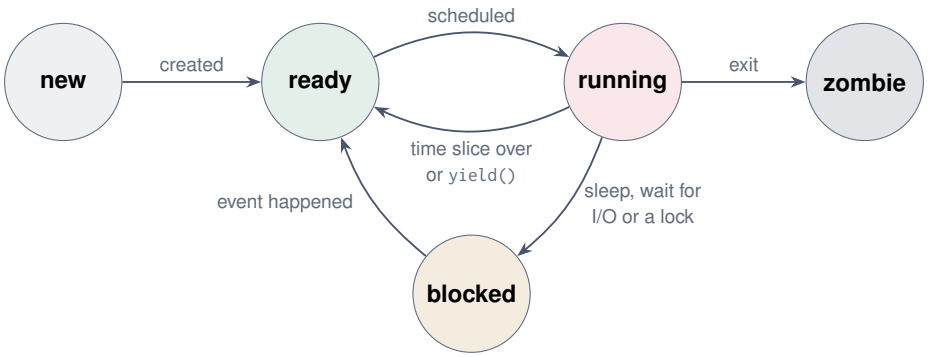


Figure 1: Task states in Kagan. Only the scheduler moves a task into *running*.

2.2 The context switch

The central point of OSTEP chapter 6 is that the kernel regains control of the CPU in only two ways: the running code calls into it (a system call, or `yield` for kernel threads), or a **timer interrupt** forces it in. Either way, the kernel is now running on the current task's kernel stack. To switch, it saves the current task's registers, records its stack pointer in its `struct task`, loads the next task's stack pointer, and restores that task's registers. Because each task has its **own kernel stack**, the part of the old task's state that lives on the stack (its return addresses, its interrupt frame) simply stays there until it is resumed.

The switch is done by a C-callable assembly function, `switch_context`. It only needs to save the four callee-saved registers (EBX, ESI, EDI, EBP) plus ESP: under cdecl, the C code that calls it has already saved anything else it cares about (Phase 1). When the new task's stack is loaded and `switch_context` executes `ret`, it returns into wherever *that* task called `switch_context` from, which might be the middle of a timer interrupt from several seconds ago (Figure 2).

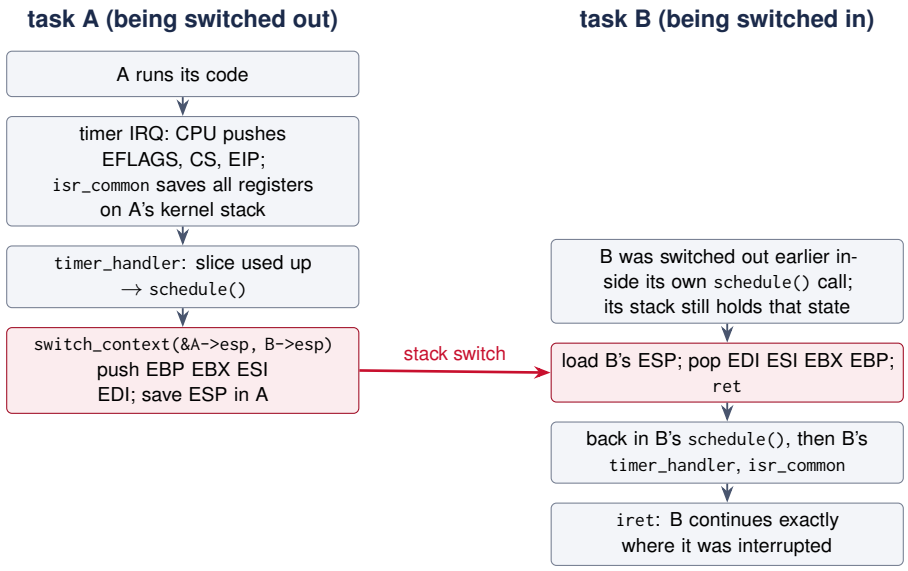


Figure 2: A preemptive switch from A to B. A's half-finished interrupt stays frozen on A's stack until A is chosen again.

2.3 Scheduling policy

With the mechanism in place, the scheduler decides who runs. OSTEP chapter 7 introduces the metrics: **turnaround time** (from arrival to completion) and **response time** (from arrival to first run). Policies like shortest-job-first optimise turnaround but need to know the future. **Round-robin** (RR) gives each ready task a fixed **time slice** (quantum) in turn, which is excellent for response time and is what Kagan starts with. Chapter 8's **multi-level feedback queue** (MLFQ) keeps several RR queues at different priorities, lowers the priority of tasks that use their whole slice (CPU-bound work), and periodically boosts everyone to prevent starvation. It favours interactive tasks without knowing anything about them in advance, which is why variants of it ran in most UNIX systems. Linux has used a third answer since 2007, the **Completely Fair Scheduler** (LKD ch. 4): no queues and no boosts, just "run the task that has had the least weighted CPU time so far". It is fairer and has fewer knobs than MLFQ, which is why the optional Task 8.9 builds it instead. OSDI §2.4.5 adds the principle to keep policy and mechanism separate, so that one can change without touching the other.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	OSTEP	Ch. 4 *The Abstraction: The Process*	25–36	Machine state, process states, the process list.
PRIMARY	OSTEP	Ch. 6 *Limited Direct Execution*, especially §6.3 *Switching Between Processes* with its timeline	52–58	The exact mechanism of Tasks 8.3–8.6.
PRIMARY	OSTEP	Ch. 7 *Scheduling: Introduction*; Ch. 8 *Scheduling: The Multi-Level Feedback Queue*	63–86	Metrics, RR, and the MLFQ rules for Task 8.9.
PRIMARY	LOB	Ch. 14 *Multitasking*: creating processes, cooperative and preemptive scheduling, separate kernel stacks	73–75	The x86 specifics, and the difficulties it warns about.
SECONDARY	MOS	§2.1 *Processes* (model, creation, states, implementation)	85–97	Another view, with the interrupt-handling sequence spelled out.
SECONDARY	MOS	§2.4 *Scheduling*	148–167	Goals per system type; RR, priority, multiple queues, lottery.
SECONDARY	OSDI	§2.5 *Overview of Processes in MINIX 3*; §2.6.5 *Process Data Structures*	112–125, 146–156	MINIX's process table and scheduling queues.
SECONDARY	OSDI	§2.6.10 *Scheduling in MINIX 3*	182–185	A real multi-queue scheduler in a few pages.
SECONDARY	OSC	§3.1–3.2 *Process Concept*, *Process Scheduling*; §5.1–5.3 *CPU scheduling*	106–116, 200–217	Queues, context switches, algorithms with examples.
OPTIONAL	OSTEP	Ch. 9 *Proportional Share*; Ch. 10 *Multiprocessor Scheduling*	87–108	Lottery and stride scheduling; what changes with more CPUs.
OPTIONAL	MOS	§2.2 *Threads*, especially §2.2.5 *Implementing Threads in the Kernel*	97–113	Kernel threads placed among the alternatives.
OPTIONAL	OSC	§5.7 *Operating-System Examples* (Linux CFS, Windows)	234–244	How production schedulers do it.
SECONDARY	LKD	Ch. 3, *Process Management*	23–40	The task structure, process states and kernel threads in Linux.
PRIMARY	LKD	Ch. 4, *Process Scheduling* (CFS, virtual runtime, wait queues, preemption)	41–67	The fair scheduler of Task 8.9.
SECONDARY	LKP	Ch. 10, *Learning about the CPU scheduling internals*, parts 2 and 3	507–528	Scheduling classes, `vruntime`, and when the scheduler runs.

Prior art: MINIX 3

MINIX keeps each process's saved registers *in the process table* (`p_reg` in `kernel/proc.h (05500)`), not on a per-process kernel stack. Its assembly entry code in `kernel/mpx386.s (06300)` saves the interrupted process's registers into its table entry, the kernel runs on one shared stack, and `restart` loads whichever process was chosen. `enqueue`, `dequeue` and `pick_proc` in `kernel/proc.c (07400)` manage 16 priority queues. This works for MINIX because its kernel never blocks in the middle of a call: work that has to wait is done by user-space servers. Kagan gives every task its

own kernel stack, like Linux and xv6, so a task can block anywhere inside the kernel (in the middle of a disk read, for example) and resume there later.

Linux, for comparison. LKD LKP Linux's normal scheduler is CFS, the Completely Fair Scheduler: each task accumulates **virtual runtime**, weighted by its priority, and the scheduler always runs the task with the smallest one, kept in a red-black tree. Round-robin and MLFQ survive only in the real-time and historical policies. Task 8.9 builds a small CFS.

4 Implementation

Task 8.1 The task structure

Define `struct task` and a global list of all tasks (for `ps`), plus a `current` pointer. Keep the saved stack pointer as the first field so the assembly never needs to know the struct layout. Hand out PIDs from a counter.

include/task.h

```
enum task_state { TASK_READY, TASK_RUNNING, TASK_BLOCKED, TASK_ZOMBIE };

struct task {
    uint32_t         esp;          /* saved kernel ESP while not running (offset 0)  */
    uint32_t         pid;
    enum task_state  state;
    char             name[16];
    void            *kstack;       /* the stack allocation, freed when reaped        */
    uint32_t         wake_tick;    /* when a sleeping task should wake               */
    int              slice;        /* ticks left in the current time slice           */
    uint32_t         cpu_ticks;    /* accounting for ps                              */
    struct list_node node;         /* on the run queue, a sleep queue, or zombies    */
    struct list_node all;          /* on the list of all tasks                       */
    uint8_t          fpu[512] __attribute__((aligned(16)));  /* fxsave area (Task 8.3) */
};

extern struct task *current;
```

Task 8.2 Kernel stacks with guard pages

Give each task a 16 KiB kernel stack in the region from `0xF0000000`, whose page tables already exist. Divide the region into 20 KiB slots and map only the top four pages of each slot, leaving the lowest page unmapped. A stack overflow then hits the guard page and produces a page fault with an address just below the stack, instead of silently overwriting the next task's memory. Keep freed slots on a list for reuse.

Verify: A test task that recurses without limit dies with a page fault whose address is in its own guard page.

Task 8.3 switch_context

Implement it as below, with a comment on every line.

kernel/arch/i386/switch.s

```
; void switch_context(uint32_t *save_esp, uint32_t load_esp)
; Saves the current task's callee-saved registers and ESP, then resumes the task
; whose saved ESP is load_esp. Must be called with interrupts disabled.
global switch_context
switch_context:
    mov  eax, [esp + 4]     ; eax = &old->esp
    mov  edx, [esp + 8]     ; edx = new->esp
```

```
    push ebp                ; save callee-saved registers on the OLD task's stack
    push ebx
    push esi
    push edi
    mov  [eax], esp         ; old->esp = current stack pointer
    mov  esp, edx           ; switch to the NEW task's stack
    pop  edi                ; restore the new task's callee-saved registers
    pop  esi
    pop  ebx
    pop  ebp
    ret                     ; return to wherever the new task last called switch_context
```

`switch_context` saves only general registers, but the CPU has more state: the eight x87 registers and, on every CPU QEMU emulates, the SSE registers. Decide now who may use them, because this is where tasks change hands. The kernel never does: add `-mgeneral-regs-only` (GCC 7 or later) to the kernel's `CFLAGS` from Phase 2, so the compiler never emits a floating-point or SSE instruction in kernel code. User programs (Phase 10) are another matter. If two of them use `double` and the kernel switches between them without saving these registers, each one computes with the other's values, and one can read what the other was working on. There are two clean policies. The first, which Kagan takes, saves and restores the state on every switch with `fxsave` and `fxrstor`, into a 512-byte area per task that must be 16-byte aligned (the `fpu` field above: `kmalloc` returns 16-byte aligned blocks, and a slab cache for `struct task` must keep its objects aligned the same way). A new task starts from the clean state that `fninit` produces. The second forbids floating point in user space: build programs with `-msoft-float`, set `CR0.EM` so that any FPU instruction raises #NM, and kill the process that executes one. Saving is the better choice, because a C library `printf("%f")` sooner or later needs it.

kernel/arch/i386/fpu.c

```
uint8_t fpu_initial[512] __attribute__((aligned(16)));   /* copied into every new task */

void fpu_init(void)                     /* once at boot; CPUID.1:EDX bit 24 (FXSR) must be set */
{
    uint32_t cr0, cr4;
    __asm__ volatile ("mov %%cr0, %0" : "=r"(cr0));
    cr0 = (cr0 & ~(1u << 2 | 1u << 3)) | 1u << 1 | 1u << 5;   /* EM=0 TS=0, MP=1 NE=1 */
    __asm__ volatile ("mov %0, %%cr0" : : "r"(cr0));
    __asm__ volatile ("mov %%cr4, %0" : "=r"(cr4));
    cr4 |= 1u << 9 | 1u << 10;          /* OSFXSR, OSXMMEXCPT: fxsave covers SSE too */
    __asm__ volatile ("mov %0, %%cr4" : : "r"(cr4));
    __asm__ volatile ("fninit; fxsave %0" : "=m"(fpu_initial));
}

void fpu_switch(struct task *prev, struct task *next)   /* from schedule() */
{
    __asm__ volatile ("fxsave %0"  : "=m"(prev->fpu));
    __asm__ volatile ("fxrstor %0" : : "m"(next->fpu));
}
```

Verify: Two test tasks each load a different value into an x87 register with inline `fld`, spin long enough to be preempted many times, store it back with `fstp` and check it; both find their own value. In Phase 10, repeat the test as two user programs running floating-point loops with `double`: both print the correct result.

Task 8.4 Creating a task

A new task has never called `switch_context`, so fake it: build its stack so the pops and `ret` land in a small trampoline. The trampoline enables interrupts (the new task doesn't return through an

interrupt path that would restore them), calls the task's entry function, and calls `task_exit` if the function ever returns.

kernel/proc/task.c

```
extern void task_trampoline(void);          /* in switch.s, see below */

struct task *task_create(const char *name, void (*entry)(void *), void *arg)
{
    struct task *t = kcalloc(1, sizeof *t);
    t->kstack = kstack_alloc();             /* 16 KiB + guard page */
    uint32_t *sp = (uint32_t *)((char *)t->kstack + KSTACK_SIZE);

    *--sp = (uint32_t)task_trampoline;      /* switch_context's ret jumps here */
    *--sp = 0;                              /* ebp                             */
    *--sp = (uint32_t)entry;                /* ebx: the function to run        */
    *--sp = (uint32_t)arg;                  /* esi: its argument               */
    *--sp = 0;                              /* edi                             */
    t->esp = (uint32_t)sp;

    memcpy(t->fpu, fpu_initial, sizeof t->fpu);   /* clean FPU state, as after fninit */
    t->pid = next_pid++;
    strncpy(t->name, name, sizeof t->name - 1);
    make_ready(t);                          /* state = READY, append to run queue */
    return t;
}
```

kernel/arch/i386/switch.s (continued)

```
extern task_exit
global task_trampoline
task_trampoline:
    sti                     ; new tasks start with interrupts enabled
    push esi                ; argument
    call ebx                ; entry(arg)
    call task_exit          ; if entry returns, end the task (never returns)
```

Verify: In GDB, break on `task_trampoline`: when it is hit, EBX holds the entry function and ESI its argument.

Portability note

Apart from the few lines that touch the interrupt flag (Phase 9 wraps them in `irq_save` and `irq_restore`), `switch_context` and the first stack frame that `task_create` builds for it are the only parts of the scheduler that depend on the CPU. Keep both in `kernel/arch/i386`: move the five `*--sp` lines into a small function next to `switch.s`, so `task.c` and `sched.c` stay plain C. Phase 16 renames the pair `arch_switch` and `arch_task_init`, and each port in Stage V writes its own.

Task 8.5 Cooperative multitasking

Turn the code that is already running (`kmain`) into task 0 by giving it a `struct task`; its `esp` gets filled in at the first switch. Write `schedule()`, which moves the current task to the back of the ready queue if it is still runnable and switches to the task at the front, and `yield()`, which calls it with interrupts disabled. Save and restore the flags rather than blindly using `sti`, because `yield` may be called with interrupts already off.

kernel/proc/sched.c

```
static struct list_node run_queue;          /* READY tasks, first in first out */

void schedule(void)                         /* interrupts must be disabled */
```

```
{
    struct task *prev = current;
    if (prev->state == TASK_RUNNING && prev != idle_task) {
        prev->state = TASK_READY;
        list_add_tail(&run_queue, &prev->node);
    }
    struct task *next = idle_task;
    if (!list_empty(&run_queue)) {
        next = container_of(run_queue.next, struct task, node);
        list_del(&next->node);
    }
    next->state = TASK_RUNNING;
    next->slice = QUANTUM_TICKS;            /* e.g. 5 ticks = 50 ms at 100 Hz */
    if (next == prev)
        return;
    current = next;
    fpu_switch(prev, next);                 /* x87/SSE registers belong to the task */
    switch_context(&prev->esp, next->esp);
}

void yield(void)
{
    uint32_t flags;
    __asm__ volatile ("pushf; pop %0; cli" : "=r"(flags));
    schedule();
    if (flags & 0x200)                      /* IF was set before: enable again */
        __asm__ volatile ("sti");
}
```

Verify: Two tasks that each print a letter and call `yield()` in a loop produce `ABABAB...`; a third task joins the rotation.

Task 8.6 Preemption from the timer

The timer takes the CPU away. In the timer handler, charge the tick to `current`, and when its slice reaches zero, call `schedule()`. One change elsewhere is essential: in `isr_handler`, send the EOI to the PIC *before* calling an IRQ handler, not after. If the handler switches to another task, the old task's half-finished interrupt stays frozen on its stack, and an EOI placed after the call would not be sent until that task runs again. Until then the PIC delivers no more timer interrupts and preemption stops. Sending it first is safe because the interrupt gate keeps interrupts disabled until `iret`.

Verify: Three tasks that loop printing without ever calling `yield` still take turns. The kernel monitor stays responsive while they run.

Task 8.7 Sleeping, idling and exiting

Add `sleep_ms`: set `wake_tick`, mark the task blocked, put it on a sleep list, and call `schedule()`. The timer handler moves every task whose `wake_tick` has passed back to the ready queue. Create an `idle` task that runs `sti; hlt` forever; `schedule` picks it only when nothing else is ready, so the CPU halts instead of spinning. Finally, `task_exit` marks the task as a zombie, puts it on a zombie list and schedules away. It can't free its own stack, because it is still running on it, so another task (the idle task is a good choice) reaps zombies: it frees their stacks and structures.

Verify: With every task sleeping, QEMU's CPU usage on the host drops to almost zero. Creating and exiting 1000 tasks leaves the PMM and heap statistics exactly where they started.

Task 8.8 ps and accounting

Add a `ps` monitor command that walks the list of all tasks and prints PID, name, state and CPU ticks. Add `spawn <name>` to start a demo task and `kill <pid>` to end one (for kernel threads, simply mark it to exit at its next tick).

Task 8.9 A fair scheduler (optional)

Round-robin gives every runnable task the same slice in turn, whatever it did before; OSTEP's MLFQ fixes that with hand-tuned queue rules. Linux's **Completely Fair Scheduler** (LKD ch. 4, LKP ch. 10) replaces both with one idea: each task accumulates **virtual runtime**, the CPU time it has used divided by its weight, and the scheduler always runs the task with the *smallest* virtual runtime. A task that slept falls behind and runs soon after it wakes; a CPU-bound task drifts to the back; a heavier weight (a higher priority) makes time pass more slowly.

Add a `vruntime` and a `weight` to `struct task`, charge the running task on every tick, and keep the run queue ordered by `vruntime`. Linux uses a red-black tree; with a few dozen tasks a sorted list is fine and keeps the scheduler short. When a task wakes, set its `vruntime` to at least the queue's minimum minus one slice, so a long sleeper cannot monopolise the CPU after waking.

kernel/proc/sched_fair.c

```
#define NICE_0_WEIGHT 1024

void sched_tick_fair(void)                    /* from the timer, before preemption */
{
    current->vruntime += (uint64_t)TICK_NS * NICE_0_WEIGHT / current->weight;
    struct task *first = runqueue_first();    /* smallest vruntime */
    if (first && first->vruntime < current->vruntime)
        need_resched = true;                  /* switch at the end of the interrupt */
}

void enqueue_fair(struct task *t)             /* keep the list sorted by vruntime */
{
    if (t->vruntime + TICK_NS < min_vruntime)
        t->vruntime = min_vruntime - TICK_NS;   /* sleepers get a small bonus, not a fortune */
    list_insert_sorted(&runqueue, t, by_vruntime);
}
```

Verify: Two CPU-bound tasks with equal weight each get about half the CPU in `ps`; give one twice the weight and it gets about two thirds. An interactive task (the monitor) still answers at once while both spin.

5 Testing and debugging

When a switch goes wrong, the symptom usually appears later, in the new task. Break on `switch_context`, step through it with `stepi`, and check with `x/6xw $esp` that the new stack holds exactly what the pops and `ret` expect. The GDB command `info symbol <addr>` shows where a return address points.

Symptom	Likely cause	Resolution
Preemption works once, then the timer stops	EOI sent after the IRQ handler, which switched away	Send the EOI before calling the handler (Task 8.6).
New task crashes at its first instruction	The initial stack doesn't match `switch_context`'s pop order	Five words: EDI, ESI, EBX, EBP, return address, from low to high.
New task runs, but never gets preempted	Interrupts disabled in the new task	The trampoline must execute `sti`.

Symptom	Likely cause	Resolution
Random corruption, other tasks' data changes	Kernel stack overflow into a neighbour	Guard pages turn this into a page fault; make stacks bigger or recursion shallower.
#GP on `fxsave` or `fxrstor`	The task's `fpu` area is not 16-byte aligned	Check the allocator's alignment, including the slab cache for `struct task`; assert `((uintptr_t)t->fpu & 15) == 0`.
Crash in `task_exit` or right after	The task freed its own stack while running on it	Only mark it as a zombie; let another task reap it.
Queue corruption under load	`schedule()` entered with interrupts enabled, so a timer interrupt re-entered it	Every path into `schedule()` disables interrupts first; assert IF is clear at its start.

6 Exit criteria

Exit criteria: Kagan multitasks

- ☐ Kernel threads run with their own 16 KiB stacks and guard pages.
- ☐ `switch_context` and `task_trampoline` work, with every instruction documented.
- ☐ The kernel is built with `-mgeneral-regs-only`, and every task has its own FPU state, saved and restored on each switch.
- ☐ At least three threads share the CPU preemptively, without calling `yield`.
- ☐ `sleep_ms` blocks properly; the idle task halts the CPU when nothing is ready.
- ☐ Tasks exit and are reaped; 1000 create/exit cycles leak no memory.
- ☐ `ps` shows every task with its state and CPU time.

Review questions

1. What state must be saved in a context switch, and why does `switch_context` save only four registers besides ESP?
2. How does the timer interrupt make preemption possible, according to OSTEP chapter 6?
3. Draw the stack of a newly created task just before its first switch.
4. Why must the EOI be sent before an IRQ handler that might call `schedule()`?
5. Round-robin versus MLFQ: what problem does MLFQ solve, and why does it need a periodic priority boost?
6. How does a vruntime scheduler choose the next task, and why must a task that slept for a minute not keep its old, small vruntime?
7. Why can't a task free its own kernel stack in `task_exit`?
8. MINIX saves registers in the process table and uses a single kernel stack. What can Kagan do that MINIX's kernel can't, and what does it cost?

7 Extensions

Implement the multi-level feedback queue of OSTEP chapter 8, or lottery or stride scheduling from chapter 9, and compare their fairness with the vruntime scheduler over a long run. Replace the sorted run queue with a red-black tree, as Linux does, and measure when it starts to pay off. Add a `top` command that shows CPU usage per task over the last second. Measure the cost of a context switch by switching back and forth a million times and reading the CPU's time-stamp counter (`rdtsc`)

before and after. For the SMP track in Phase 15, OSTEP chapter 10 describes what has to change: one `current` per CPU, per-CPU run queues, and real locks.

Next: **Phase 9 Synchronization and Concurrency in the Kernel** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.